In [75]:
```
import pandas as pd
import numpy as np

pd.set_option('display.max_columns',None)
```

In [76]:
```
ds=pd.read_csv('phishingtwo.csv')
ds.shape
```

Out[76]:
```
(11054, 32)
```

In [77]:
```
ds.columns
```

Out[77]:
```
Index(['Index', 'UsingIP', 'LongURL', 'ShortURL', 'Symbol@', 'Redirecting//',
       'PrefixSuffix-', 'SubDomains', 'HTTPS', 'DomainRegLen', 'Favicon',
       'NonStdPort', 'HTTPSDomainURL', 'RequestURL', 'AnchorURL',
       'LinksInScriptTags', 'ServerFormHandler', 'InfoEmail', 'AbnormalURL',
       'WebsiteForwarding', 'StatusBarCust', 'DisableRightClick',
       'UsingPopupWindow', 'IframeRedirection', 'AgeofDomain', 'DNSRecording',
       'WebsiteTraffic', 'PageRank', 'GoogleIndex', 'LinksPointingToPage',
       'StatsReport', 'class'],
      dtype='object')
```

In [78]:
```
ds= ds[['UsingIP', 'LongURL', 'ShortURL', 'Symbol@', 'Redirecting//',
       'PrefixSuffix-', 'SubDomains', 'HTTPS', 'DomainRegLen', 'Favicon',
       'NonStdPort', 'HTTPSDomainURL', 'RequestURL', 'AnchorURL',
       'LinksInScriptTags', 'ServerFormHandler', 'InfoEmail', 'AbnormalURL',
       'WebsiteForwarding', 'StatusBarCust', 'DisableRightClick',
       'UsingPopupWindow', 'IframeRedirection', 'AgeofDomain', 'DNSRecording',
       'WebsiteTraffic', 'PageRank', 'GoogleIndex', 'LinksPointingToPage',
       'StatsReport', 'class']]
```

In [79]:
```
ds.head()
```

Out[79]:

	UsingIP	LongURL	ShortURL	Symbol@	Redirecting//	PrefixSuffix-	SubDomains	H
0	1	1	1	1	1	-1	0	
1	1	0	1	1	1	-1	-1	
2	1	0	1	1	1	-1	-1	
3	1	0	-1	1	1	-1	1	
4	-1	0	-1	1	-1	-1	1	

In [80]:
```
# Map -1 → 0 for XGBoost compatibility
y_train_xgb = np.where(y_train == -1, 0, 1)
y_test_xgb  = np.where(y_test  == -1, 0, 1)

print("Mapped train classes:", np.unique(y_train_xgb))
print("Mapped test classes: ", np.unique(y_test_xgb))
```

```
Mapped train classes: [0 1]
Mapped test classes:  [0 1]
```

In [63]:
```
ds.isna().sum()
```

Out[63]:
```
UsingIP                0
LongURL                0
ShortURL               0
Symbol@                0
Redirecting//          0
PrefixSuffix-          0
SubDomains             0
HTTPS                  0
DomainRegLen           0
Favicon                0
NonStdPort             0
HTTPSDomainURL         0
RequestURL             0
AnchorURL              0
LinksInScriptTags      0
ServerFormHandler      0
InfoEmail              0
AbnormalURL            0
WebsiteForwarding      0
StatusBarCust          0
DisableRightClick      0
UsingPopupWindow       0
IframeRedirection      0
AgeofDomain            0
DNSRecording           0
WebsiteTraffic         0
PageRank               0
GoogleIndex            0
LinksPointingToPage    0
StatsReport            0
class                  0
dtype: int64
```

In [81]:
```
xds= ds[['UsingIP', 'LongURL', 'ShortURL', 'Symbol@', 'Redirecting//',
       'PrefixSuffix-', 'SubDomains', 'HTTPS', 'DomainRegLen', 'Favicon',
       'NonStdPort', 'HTTPSDomainURL', 'RequestURL', 'AnchorURL',
       'LinksInScriptTags', 'ServerFormHandler', 'InfoEmail', 'AbnormalURL',
       'WebsiteForwarding', 'StatusBarCust', 'DisableRightClick',
       'UsingPopupWindow', 'IframeRedirection', 'AgeofDomain', 'DNSRecording',
       'WebsiteTraffic', 'PageRank', 'GoogleIndex', 'LinksPointingToPage',
       'StatsReport']]

yds = ds[['class']]
```

In [82]:
```
xds
```

Out[82]:

	UsingIP	LongURL	ShortURL	Symbol@	Redirecting//	PrefixSuffix-	SubDomai
0	1	1	1	1	1	-1	
1	1	0	1	1	1	-1	
2	1	0	1	1	1	-1	
3	1	0	-1	1	1	-1	
4	-1	0	-1	1	-1	-1	
...	...	...	...	...	...	...	
11049	1	-1	1	-1	1	1	
11050	-1	1	1	-1	-1	-1	
11051	1	-1	1	1	1	-1	
11052	-1	-1	1	1	1	-1	
11053	-1	-1	1	1	1	-1	

11054 rows × 30 columns

In [83]:
```
yds
```

Out[83]:

	class
0	-1
1	-1
2	-1
3	1
4	1
...	...
11049	1
11050	-1
11051	-1
11052	-1
11053	-1

11054 rows × 1 columns

In [67]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(xds,yds, test_size=0.20,random_

print("Train:", x_train.shape, "| Test:", x_test.shape)
print("\nTrain class balance:")
print(y_train.value_counts(normalize=True))
```

```
Train: (8843, 30) | Test: (2211, 30)

Train class balance:
class
 1       0.556938
-1       0.443062
Name: proportion, dtype: float64
```

In [68]:
```
y_test.value_counts(normalize=True)
```

Out[68]:
```
class
 1       0.557214
-1       0.442786
Name: proportion, dtype: float64
```

In [69]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

In [70]:
```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree":       DecisionTreeClassifier(random_state=42),
    "Random Forest":       RandomForestClassifier(n_estimators=100, random_state
    "XGBoost":             XGBClassifier(eval_metric='logloss', random_state=42)
}

trained_models = {}

for name, model in models.items():
    print(f"Training {name}...")
    if name == "XGBoost":
        model.fit(x_train, y_train_xgb)
    else:
        model.fit(x_train, y_train)
    trained_models[name] = model
    print(f"  Done.")
```

```
Training Logistic Regression...
  Done.
Training Decision Tree...
  Done.
Training Random Forest...
  Done.
Training XGBoost...
  Done.
```

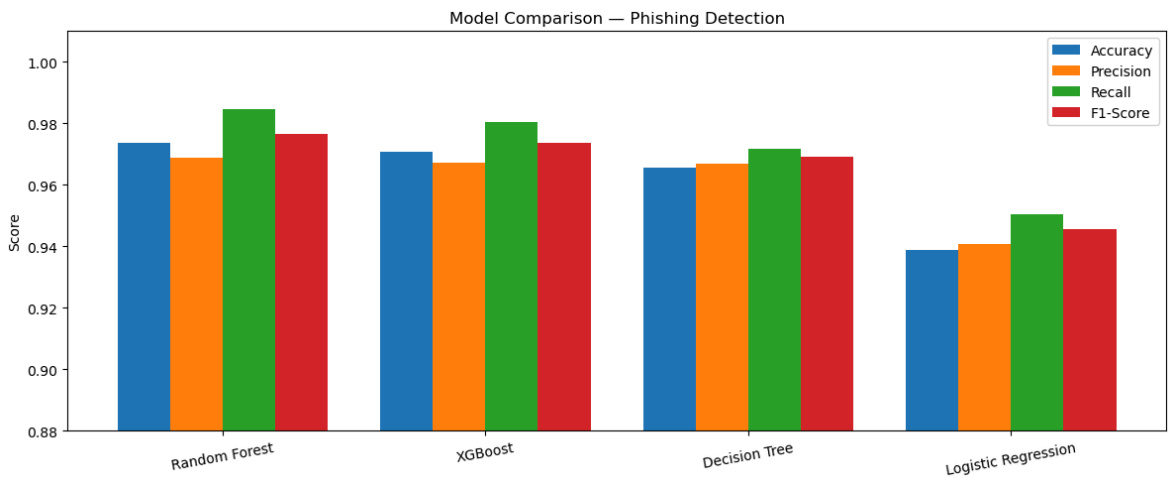
In [71]:
```
results = []

for name, model in trained_models.items():
    if name == "XGBoost":
```

```
        y_pred_raw = model.predict(x_test)
        # Map back: 0 → -1, 1 → 1
        y_pred = np.where(y_pred_raw == 0, -1, 1)
    else:
        y_pred = model.predict(x_test)

    results.append({
        "Model":     name,
        "Accuracy":  round(accuracy_score(y_test, y_pred), 4),
        "Precision": round(precision_score(y_test, y_pred, pos_label=1), 4),
        "Recall":    round(recall_score(y_test, y_pred, pos_label=1), 4),
        "F1-Score":  round(f1_score(y_test, y_pred, pos_label=1), 4)
    })

results_df = pd.DataFrame(results).sort_values("F1-Score", ascending=False)
print(results_df.to_string(index=False))
```

```
              Model  Accuracy  Precision  Recall  F1-Score
      Random Forest    0.9738     0.9688  0.9846    0.9767
            XGBoost    0.9706     0.9672  0.9805    0.9738
      Decision Tree    0.9656     0.9669  0.9716    0.9692
Logistic Regression    0.9389     0.9406  0.9505    0.9455
```

In [72]:

```
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
x = np.arange(len(results_df))
width = 0.2

fig, ax = plt.subplots(figsize=(12, 5))
for i, metric in enumerate(metrics):
    ax.bar(x + i * width, results_df[metric], width, label=metric)

ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(results_df["Model"], rotation=10)
ax.set_ylim(0.88, 1.01)
ax.set_ylabel("Score")
ax.set_title("Model Comparison — Phishing Detection")
ax.legend()
plt.tight_layout()
plt.show()
```

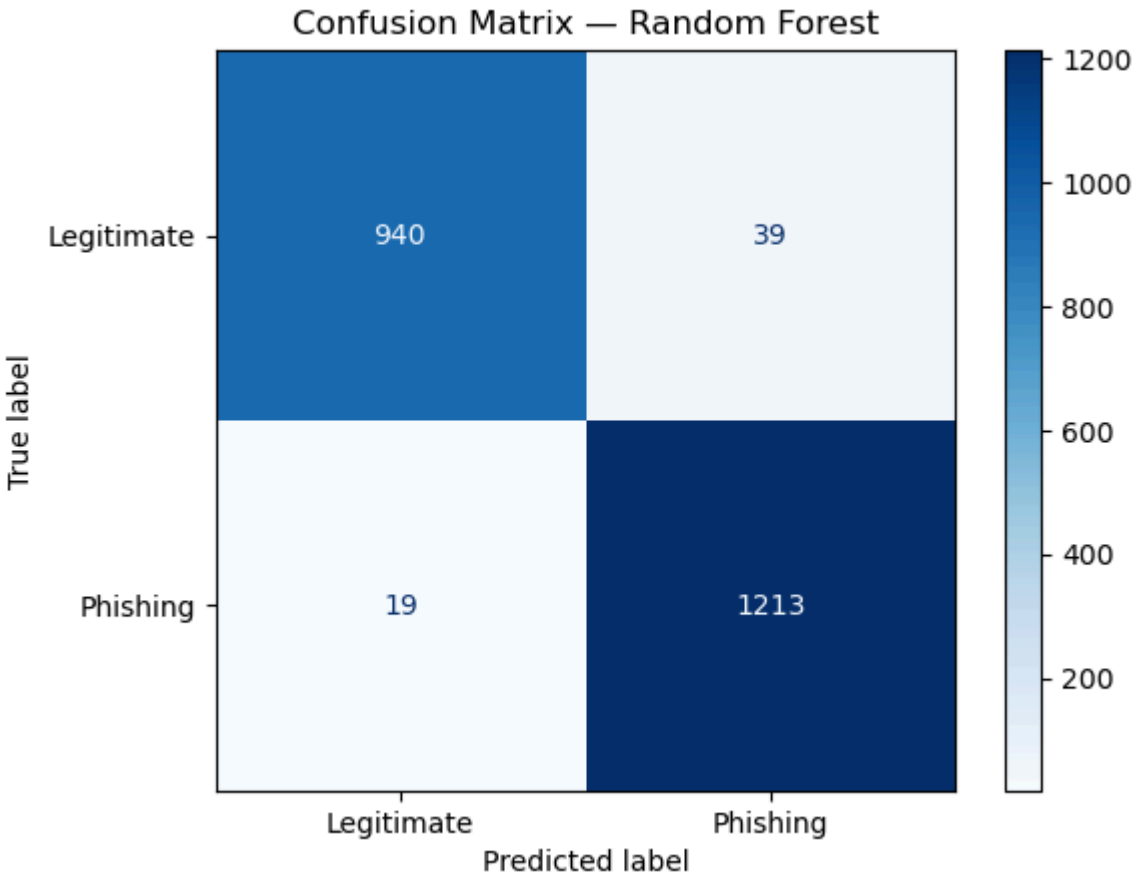


In [73]:

```
best_name = results_df.iloc[0]["Model"]   # Random Forest
best_model = trained_models[best_name]
y_pred_best = best_model.predict(x_test)

print(f"=== Best Model: {best_name} ===\n")
```

```
print(classification_report(y_test, y_pred_best,
                            target_names=["Legitimate (-1)", "Phishing (1)"]))

cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(cm, display_labels=["Legitimate", "Phishing"])
disp.plot(cmap="Blues")
plt.title(f"Confusion Matrix — {best_name}")
plt.show()
```

```
=== Best Model: Random Forest ===

                 precision    recall  f1-score   support

Legitimate (-1)       0.98      0.96      0.97       979
   Phishing (1)       0.97      0.98      0.98      1232

       accuracy                           0.97      2211
      macro avg       0.97      0.97      0.97      2211
   weighted avg       0.97      0.97      0.97      2211
```



In [74]:
```
import joblib
joblib.dump(best_model, "best_model_Random_Forest.pkl")
print("Model saved successfully!")
```

```
Model saved successfully!
```

In [86]:
```
!pip install shap
```

```
Requirement already satisfied: shap in c:\users\kpash\anaconda3\lib\site-packages
(0.50.0)
Requirement already satisfied: numpy>=2 in c:\users\kpash\anaconda3\lib\site-pack
ages (from shap) (2.3.5)
Requirement already satisfied: scipy in c:\users\kpash\anaconda3\lib\site-package
s (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in c:\users\kpash\anaconda3\lib\site-
packages (from shap) (1.7.2)
Requirement already satisfied: pandas in c:\users\kpash\anaconda3\lib\site-packag
es (from shap) (2.3.3)
Requirement already satisfied: tqdm>=4.27.0 in c:\users\kpash\anaconda3\lib\site-
packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in c:\users\kpash\anaconda3\lib\sit
e-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in c:\users\kpash\anaconda3\lib\site
-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in c:\users\kpash\anaconda3\lib\site-p
ackages (from shap) (0.62.1)
Requirement already satisfied: cloudpickle in c:\users\kpash\anaconda3\lib\site-p
ackages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in c:\users\kpash\anaconda3\lib
\site-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.46,>=0.45.0dev0 in c:\users\kpash\anaco
nda3\lib\site-packages (from numba>=0.54->shap) (0.45.1)
Requirement already satisfied: colorama in c:\users\kpash\anaconda3\lib\site-pack
ages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\kpash\anaconda3
\lib\site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\kpash\anaconda3\lib\site-
packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\kpash\anaconda3\lib\sit
e-packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\kpash\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\kpash\anaconda3\lib\site
-packages (from scikit-learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\kpash\anaconda3\l
ib\site-packages (from scikit-learn->shap) (3.5.0)
```

In [92]:

```python
import shap
import matplotlib.pyplot as plt

# Use XGBoost model instead - SHAP is natively optimized for it
xgb_model = trained_models["XGBoost"]

# Only 100 samples
x_shap = x_test.iloc[:100]
x_shap_xgb = np.where(x_shap == -1, 0, 1)  # convert labels for XGBoost

explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(x_shap_xgb)

print("SHAP done! Shape:", shap_values.shape)
```

```
SHAP done! Shape: (100, 30)
```

In [94]:

```python
# Get feature names from your dataset
feature_names = [col for col in ds.columns if col != 'class']
print("Features:", feature_names)
```

```
Features: ['UsingIP', 'LongURL', 'ShortURL', 'Symbol@', 'Redirecting//', 'PrefixS
uffix-', 'SubDomains', 'HTTPS', 'DomainRegLen', 'Favicon', 'NonStdPort', 'HTTPSDo
mainURL', 'RequestURL', 'AnchorURL', 'LinksInScriptTags', 'ServerFormHandler', 'I
nfoEmail', 'AbnormalURL', 'WebsiteForwarding', 'StatusBarCust', 'DisableRightClic
k', 'UsingPopupWindow', 'IframeRedirection', 'AgeofDomain', 'DNSRecording', 'Webs
iteTraffic', 'PageRank', 'GoogleIndex', 'LinksPointingToPage', 'StatsReport']
```

In [ ]:

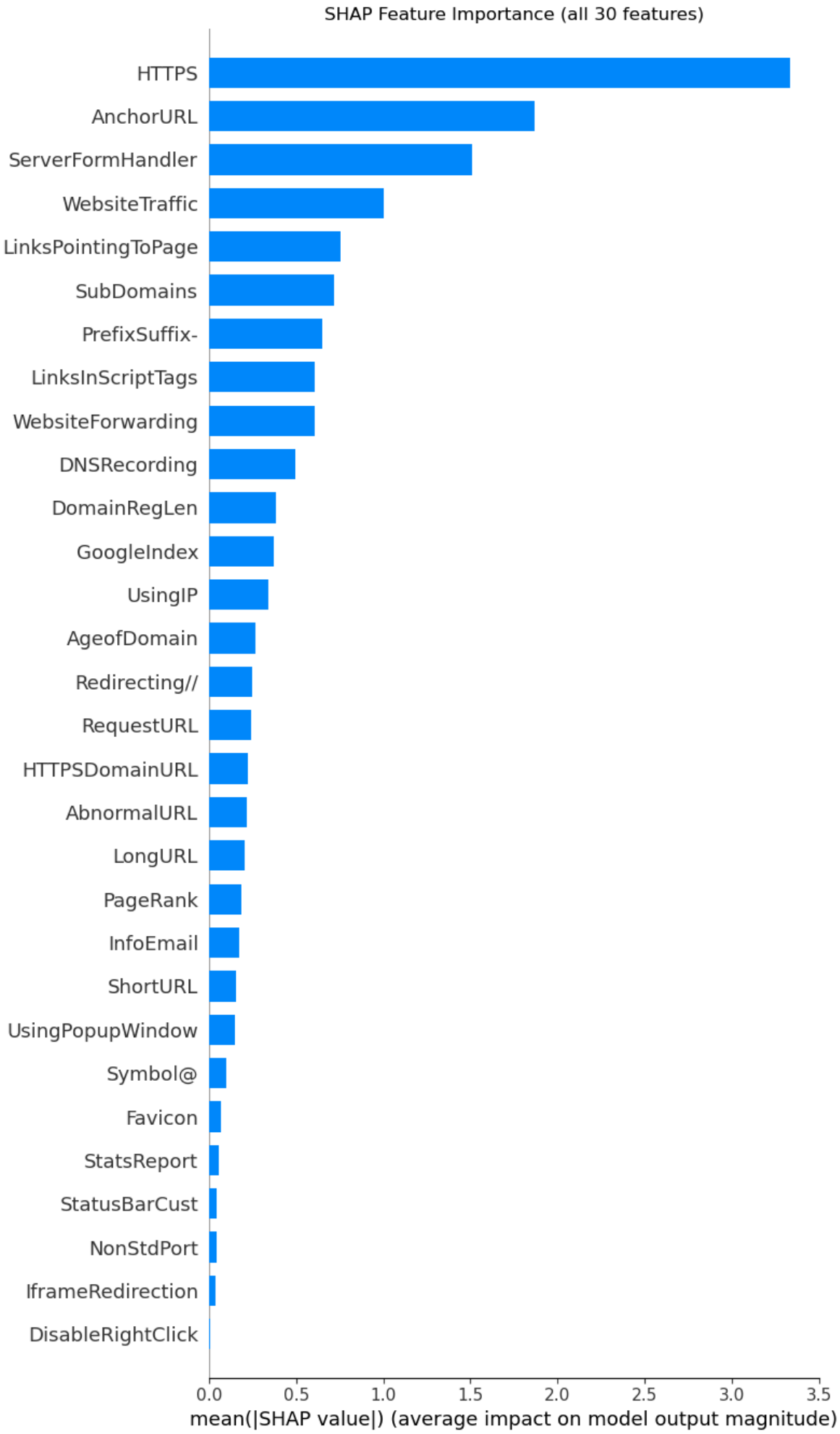
In [98]:

```python
import pandas as pd

# Get mean absolute SHAP value for each feature
shap_importance = pd.DataFrame({
    'Feature': feature_names,
    'SHAP Importance': abs(shap_values).mean(axis=0)
}).sort_values('SHAP Importance', ascending=False)

print("=== All 30 Features with SHAP Importance ===\n")
print(shap_importance.to_string(index=False))

print(f"\nFeatures with zero importance: {(shap_importance['SHAP Importance'] ==
print(f"Features that contributed:     {(shap_importance['SHAP Importance'] > 0)
```

```
=== All 30 Features with SHAP Importance ===

            Feature  SHAP Importance
              HTTPS         3.335327
          AnchorURL         1.867089
  ServerFormHandler         1.509689
     WebsiteTraffic         1.001808
LinksPointingToPage         0.758130
         SubDomains         0.720959
      PrefixSuffix-         0.650547
  LinksInScriptTags         0.609065
  WebsiteForwarding         0.605866
       DNSRecording         0.495360
       DomainRegLen         0.383720
        GoogleIndex         0.373358
            UsingIP         0.344071
        AgeofDomain         0.267754
      Redirecting//         0.250492
         RequestURL         0.241321
     HTTPSDomainURL         0.225573
        AbnormalURL         0.218018
            LongURL         0.208043
           PageRank         0.184768
          InfoEmail         0.172978
           ShortURL         0.154117
   UsingPopupWindow         0.151540
            Symbol@         0.099503
            Favicon         0.066935
        StatsReport         0.056826
      StatusBarCust         0.042252
         NonStdPort         0.041163
  IframeRedirection         0.038758
  DisableRightClick         0.004405

Features with zero importance: 0
Features that contributed:     30
```

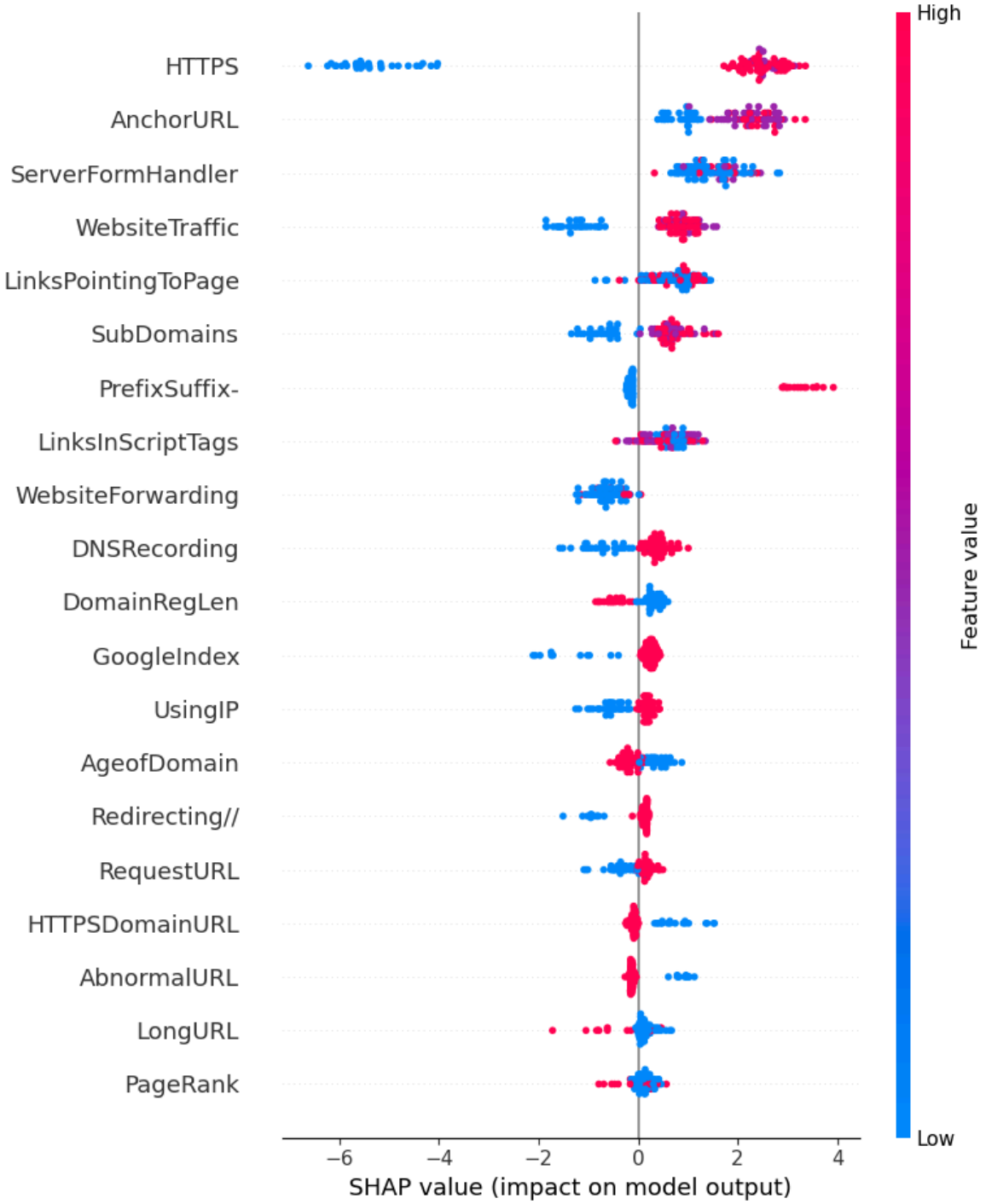
In [99]:

```
shap.summary_plot(shap_values, x_shap,
                  feature_names=feature_names,
                  plot_type="bar",
                  max_display=30,      # show all 30
                  show=False)
plt.title("SHAP Feature Importance (all 30 features)")
plt.tight_layout()
plt.savefig("shap_bar_all.png", dpi=150, bbox_inches='tight')
plt.show()
```


SHAP Feature Importance (all 30 features)
HTTPS
AnchorURL
ServerFormHandler
WebsiteTraffic
LinksPointingToPage
SubDomains
PrefixSuffix-
LinksInScriptTags
WebsiteForwarding
DNSRecording
DomainRegLen
GoogleIndex
UsingIP
AgeofDomain
Redirecting//
RequestURL
HTTPSDomainURL
AbnormalURL
LongURL
PageRank
InfoEmail
ShortURL
UsingPopupWindow
Symbol@
Favicon
StatsReport
StatusBarCust
NonStdPort
IframeRedirection
DisableRightClick
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
mean(|SHAP value|) (average impact on model output magnitude)

In [100…

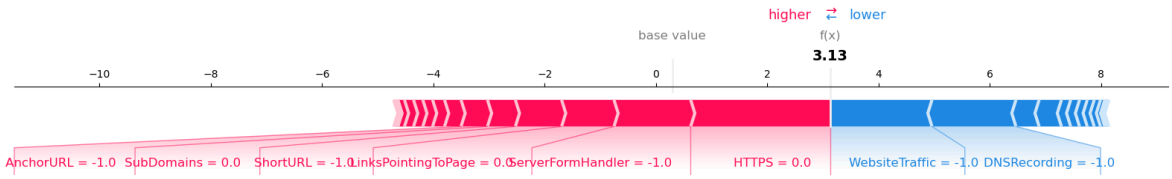
```
shap.summary_plot(shap_values, x_shap,
                  feature_names=feature_names,
                  show=False)
plt.tight_layout()
plt.savefig("shap_beeswarm.png", dpi=150, bbox_inches='tight')
plt.show()
```



In [101…

```
shap.force_plot(
    explainer.expected_value,
    shap_values[0],
    x_shap.iloc[0],
    feature_names=feature_names,
    matplotlib=True,
    show=False
)
```

```
plt.savefig("shap_force.png", dpi=150, bbox_inches='tight')
plt.show()
```



In [102…

```
shap_importance = pd.DataFrame({
    'Feature': feature_names,
    'SHAP Importance': abs(shap_values).mean(axis=0)
}).sort_values('SHAP Importance', ascending=False)

print("=== Top 10 Most Important Features ===\n")
print(shap_importance.head(10).to_string(index=False))
```

```
=== Top 10 Most Important Features ===

            Feature  SHAP Importance
              HTTPS         3.335327
          AnchorURL         1.867089
  ServerFormHandler         1.509689
     WebsiteTraffic         1.001808
LinksPointingToPage         0.758130
         SubDomains         0.720959
      PrefixSuffix-         0.650547
  LinksInScriptTags         0.609065
  WebsiteForwarding         0.605866
       DNSRecording         0.495360
```

In [103…

```
import joblib
joblib.dump(best_model, "best_model_Random_Forest.pkl")
joblib.dump(trained_models["XGBoost"], "xgb_model.pkl")
print("Models saved!")
```

```
Models saved!
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: